

Quantitative Methods for Humanities

Topic 0.5 (optional): Introduction to R

2026/2027

General information

- ▶ This deck is optional material for students who want to use R/RStudio as a quantitative tool.
- ▶ Required assessment focuses on course concepts; it does not require students to write R code.
- ▶ R may still appear in instructor-led demonstrations and case studies.

- ▶ Prerequisites:
 - Linear algebra (school level)
 - Statistics (school level)
- ▶ Laptop (any standard operating system) with internet access

Essential: W. John Braun and Duncan J. Murdoch
A First Course in Statistical Programming with R
Third Edition. Cambridge University Press.

Additional: Wickham, H. and Grolemond, G.
R for Data Science, O'Reilly, 2016.

About R

- ▶ Open source programming language
- ▶ Available for Windows, Linux, and MacOS
- ▶ Used by leading companies like Google, Pfizer, Merck, Bank of America, the InterContinental Hotels Group, Shell, ...
- ▶ Internet site: www.r-project.org
- ▶ Comprehensive R Archive Network (CRAN): cran.at.r-project.org

“

The great beauty of R is that you can modify it to do all sorts of things, and you have a lot of prepackaged stuff that's already available, so you're standing on the shoulders of giants.

- Hal Varian, chief economist at Google.

”

- ▶ Open www.r-project.org in your browser
- ▶ In the left menu, choose CRAN (or click "download R")
- ▶ Choose a mirror (e.g., Spain)
- ▶ Choose your operating system (Windows or macOS)
- ▶ Download and install:
 - Windows: choose base, download the newest version, run the .exe
 - macOS: download the newest .pkg installer and open it
- ▶ Start R

Command window (R Console) Prompt: >

You can input commands and execute them (by pressing the RETURN key).

Examples

```
> 1+1  
> 1+1 # This is a comment: 1+1  
> (1+2)*3  
> (5/3)^4.5  
> 5+2; 7+3; 2*5
```


Concatenation: `c()`

Assignment operator: `<-` or `=`

Examples

```
> c(1,4,7)
> a <- c(1,4,7)
> print(a)
> a
> A
> b <- c(1,a,3)
> b
> mean(b)
```

Quitting

- ▶ Quit R by the command `q()`
- ▶ In general, do *not* save your workspace

- ▶ Long computations should not be done in the command window
- ▶ Use an editor to write a program and then execute it in R
- ▶ There is a built-in editor in R: File – New script
- ▶ External editors:
 - **R-Studio**, <https://posit.co/downloads/>, choose RStudio Desktop FREE
 - Tinn-R, <http://sciviews.org/Tinn-R/>
 - Notepad++: <http://www.notepad-plus-plus.org/>
- ▶ We will focus on **R-Studio**, by far the most popular one.

▶ RStudio Cloud

- Open <https://www.rstudio.com/products/cloud/>
- Click on GET STARTED (for free) and register (sign up)
- Start a new project

▶ RStudio on your computer

- Open <https://posit.co/download/rstudio-desktop/>
- Skip the Download-and-install-R step (you already did it)
- Download **RStudio Desktop** (free, Open Source edition) for your operating system
- Run the installer (.exe on Windows, .dmg on macOS) and follow the instructions
- Start RStudio

- ▶ Instead of typing in commands in the console, **create scripts and save your work**
- ▶ You will be able to comment, edit, and recompile your code
- ▶ In R-Studio: top left corner R script
- ▶ Type a few lines of commands, e.g.

```
a <- c(1,4,7)  
mean(a)  
mean(a)^2
```
- ▶ **Execute a single line** by pressing CTRL-ENTER
- ▶ **Execute multiple lines/parts** by marking them and then pressing CTRL-ENTER
- ▶ Save the script, quit R, restart R, open the script and execute it

Style Guide (by Hadley Wickham)

1. Use meaningful file names ending in `.R` (e.g., `cleaning.R`, `regression.R`)
2. Use `lowercase_with_underscores` for names (e.g., `turnout_rate`, `load_data()`)
3. Variable names and commands are case-sensitive (e.g., `mean` $\neq$ `Mean`; `x` $\neq$ `X`)
4. Names may include letters, numbers (not first), and dots (e.g., `x2`, `party.name`; not `2x`)
5. Don't reuse names of existing functions/objects
6. Put spaces around operators (e.g., `x <- 1`, `y = x + 2`)
7. Put a space after commas, never before (e.g., `c(1, 2, 3)`)
8. No spaces inside `()` or `[]` (except after commas): `mean(x)`, `x[1, 2]`
9. Keep lines $\leq$ 80 characters (split long lines if needed)
10. Use `<-` (not `=`) for assignment: `x <- 1`
11. Comment your code: start with `#` and keep comments helpful, not excessive

Basics

- ▶ To obtain details about a command, type
`?command_name`
or
`help(command_name)`
- ▶ Example: `?mean` or `help(mean)`
- ▶ In R-Studio: bottom right corner

One of the strengths of R is the large and growing collection of packages that can be installed from CRAN (The Comprehensive R Archive Network)

- ▶ **Installation:** `install.packages('package_name')`
- ▶ **Activation:** `library(package_name)`
- ▶ **Help:** `library(help=package_name)`
- ▶ In R-Studio: bottom right corner

- ▶ R is object-oriented
- ▶ Objects store data, functions, results of operations, ...
- ▶ Everything in R is an object (e.g., scalar, vector, matrix, string)
- ▶ Every object has a unique (case-sensitive) name
- ▶ Examples:
 - Vectors:
`x <- c(1, 2, 3)`
 - Matrices:
`m <- matrix(1:6, nrow = 2, ncol = 3)`
 - Data Frames:
`df <- data.frame(Name = c("Alice", "Bob"), Age = c(25, 30))`

- ▶ Classes define the type and behavior of an object.
- ▶ Determines how an object responds to functions.
- ▶ Common classes:
 - **Numeric:** Numbers (e.g., 3.14, 42).
 - **Character:** Text strings (e.g., "Hello").
 - **Logical:** Boolean values (TRUE, FALSE).
 - **Factors:** Categorical data with levels.
 - **Lists:** Collections of objects.
- ▶ Examples:
 - **Numeric class:**

```
num <- 42  
class(num)
```
 - **Factor class:**

```
gender <- factor(c("male", "female", "female"))  
class(gender)
```

▶ Functions in R perform specific tasks or operations.

▶ **Syntax:** `func_name(input)`

▶ Some basic functions:

- `sqrt()` - Square root.
- `print()` - Print output.
- `c()` - Combine elements into a vector.
- `length()` - Number of elements in a vector.
- `min()` - Minimum value.
- `max()` - Maximum value.
- `range()` - Range of values.
- `mean()` - Average of values.
- `sum()` - Total of values.

▶ Example (mean world population from 1950 to 2010):

```
world.pop <- c(2525779, 3026003, 3691173, 4449049, 5320817, 6127700, 6916183)
mean(world.pop)
```

Functions with multiple arguments

- ▶ **Syntax:** `funcname(arg1 = input1, arg2 = input2)`
- ▶ **Example:** `seq()` function
 - Generates a sequence of numbers.
 - Arguments:
 - `from` - start value
 - `to` - end value
 - `by` - interval step value
- ▶ **Implementation:** `seq(from = 1950, to = 2010, by = 10)`

- ▶ The working directory is where R reads and saves files.

- ▶ Check the current directory:

```
getwd()
```

- ▶ Set a new directory via the console:

```
setwd("path/to/folder")
```

- ▶ Set a new directory via the menu bar:

Session → Set Working Directory →:

- To Source File Location (current file path)
- Choose Directory... (custom path)

- ▶ **CSV files:** tabular data with observations separated by line breaks and variables within the observation separated by a comma, a tab, or other characters.
- ▶ **Read CSV files:** `read.csv(path/to/file)`
- ▶ Other formats and functions to read them:
 - Excel Files: `read_excel("file.xlsx")` (requires `readxl` package)
 - RData Files: `load("file.RData")`

- ▶ **Storing Data:** typically stored in `data.frame` objects
- ▶ **Summary function:**
 - `names()` - Names of the variables (columns)
 - `nrow()` - Number of rows
 - `ncol()` - Number of columns
 - `dim()` - Dimensions
 - `summary()` - Summary statistics

Accessing variables and observations

Imaging that the object `UN.pop` is a `data.frame` containing 7 observations (rows) of the variables (columns) `year` and `world.pop`. You can access specific variables and observations like this:

- ▶ **Using the \$ operator:**

`UN.pop$world.pop` - extract the `world.pop` variable

- ▶ **Using []:**

`UN.pop[, "world.pop"]` - extract the `world.pop` column

`UN.pop[c(1, 2, 3), ]` - extract the first three rows (all columns)

`UN.pop[1:3, "year"]` - extract the first three values of the `year` column

- ▶ **Combining techniques to retrieve specific elements:**

`UN.pop$world.pop[seq(from = 1, to = nrow(UNpop), by = 2)]` - retrieve odd elements of the `world.pop` variable

Exercises

Exercise 1

Install the packages MASS, devtools, lintr.

The current working directory (where R reads and writes files) can be found by the command `getwd()`. Find your current working directory.

Exercise 3

Use the command `setwd()` to change the working directory to the path "C:/Users/Public/Documents". Note that the path name is structured by slashes (/), *not* backslashes (\). Now change the working directory to "C:/Users/Public/Downloads" via the menu bar.

Exercise 4

Open a new script file. Type the commands to perform the following assignments:

$$a = \frac{3 \cdot (4 + 9)}{8 - 12.5}$$

$$b = (1, 4, 1999, 2011)$$

$$d = 2\pi$$

$$e = a^2 + \sqrt{d}$$

Save the script with the name `Basic_R` in a path of your choice (you can create a folder called `QuantMeth` or something similar to start saving there the course material) and quit R.

Exercise 5

Start R and re-open the script. Set the working directory to the path you just created in the previous part (and where the R script is) by using the “To Source File Location” option in the menu bar. Mark all lines (CTRL + A) and execute them (CTRL + ENTER). Mark one line and execute it. Mark a part of a line and execute it. Print *a*, *b*, *d*, *e*. Why is the variable name *c* not used?

Exercise 6

In the same R script file, solve the following problem. Add comments to explain your code and to delimit the code regions of this and the previous exercise.

- ▶ Use the function `seq()` to create a vector named `year` with the values from 2010 to 1950 with a interval step of -10 .
- ▶ Print the vector using the function `print()`.
- ▶ Use the following functions on the vector `year`
 - `length()`
 - `min()`
 - `max()`
 - `range()`
 - `mean()`
 - `sum()`

Exercise 7

- ▶ Download the [UNpop.csv](#) CSV file, which contains the world population in several years according to the United Nations.
- ▶ Save the file along with the R script `Basic_R`, in the same path of the script called `Basic_R`.
- ▶ Import the data contained in the CSV file and store it in an object called `UNpop`.
- ▶ Check the class of `UNpop`.
- ▶ Use the functions `names()`, `nrow()`, `ncol()`, `dim()`, and `summary()` on the object `UNpop`. Comment on the results.
- ▶ Extract the column called `year` from `UNpop`, both using the `$` operator and the `[ ]` option.
- ▶ Extract the 4th and 5th observations of both columns.
- ▶ Extract the even observations of the `world.pop` variable and save them in a vector called `even.world.pop`.
- ▶ Load the same data in RData format, which you can download from [here](#).

The `lintr` package (already installed in Exercise 1) helps in correcting the coding style and syntax of an R script aligning it to a well-established style guidelines.

- ▶ Load the `lintr` package using the command `library(lintr)`
- ▶ Run the command `lintr("Basic_R.R")` (provided your working directory contains the script) to see the recommended style changes.
- ▶ Implement the changes and save the script.

R Markdown

- ▶ Formatting tool for creating scientific reports
- ▶ Includes embedded R code chunks
- ▶ File extension: .Rmd
- ▶ Internet site: <https://rmarkdown.rstudio.com/>
- ▶ Cheat Sheet: <https://rstudio.github.io/cheatsheets/rmarkdown.pdf>

Exercise 10

- ▶ Create an HTML format Markdown file (see cheat sheet) called "My first Markdown"
- ▶ Use the "Knit" button to compile
- ▶ Use the shortcut Ctrl + Shift + K to compile
- ▶ Save your Markdown file
- ▶ Create your own Markdown report for the solution of Exercises 4, 6, and 7
- ▶ Try the option `{r echo=FALSE}`
- ▶ Create a more polished report-like HTML document. (try to introduce title, author, separated sections for each exercise, ...)